```
    case "clearHolds": {
      return { ...state, myTickets: [] }
    }
    default:
      return state
  }
}
```

What's interesting is that basically none of this is React-specific.

We define two methods. The first, initialState, creates an initial state from the incoming props passed to the app. The return type of the state is called VenueState, and it is defined in the venue_types file.

The second method is venueReducer, which is the reducer function itself. All of our business logic has been reworked to match the reducer pattern—which is to say, the logic has been reworked to use the state coming into the reducer function and return a new state object with the changes.

There are five actions in the venueReducer, all of which return a new state object. They all heavily use the JavaScript spread operator, which makes it easy to say that an object should be copied but specific fields should be overridden. The JS { ...state, ticketsToBuy: action.amount } means create a new object with the same attributes as the existing state but override ticketsToBuy with the action.amount.

These are the five actions:

- setTickets, called after we fetch the ticket data from the server, and it passes that data to myTickets, a list of tickets held by the user, and otherTickets, a list of tickets held by others
- setTicketsToBuy, called when the number of tickets in the pull-down menu changes, updating the ticketsToBuy in the state
- holdTicket, called when the user clicks on an available seat, which generates ticket data for multiple tickets based on the ticketsToBuy count, then adds that data to the myTickets